

Del encargo al informe técnico

La unidad que junta todo lo anterior: modelar, construir, asegurar, y además documentarlo de principio a fin.

Rubén Schnettler • Docente

INACAP Valparaíso • Analista Programador

BLOQUE 6

BLOQUE

01

Ingeniería de software y metodologías ágiles

Por qué construir software es una disciplina de ingeniería, y cómo las metodologías ágiles organizan ese trabajo en la industria.

INGENIERÍA DE SOFTWARE

Programar no es lo mismo que hacer ingeniería

La definición de referencia

«La aplicación de un enfoque **sistemático, disciplinado y cuantificable** al desarrollo, la operación y el mantenimiento del software; es decir, la aplicación de la ingeniería al software.»

IEEE Std 610.12-1990, glosario de ingeniería de software

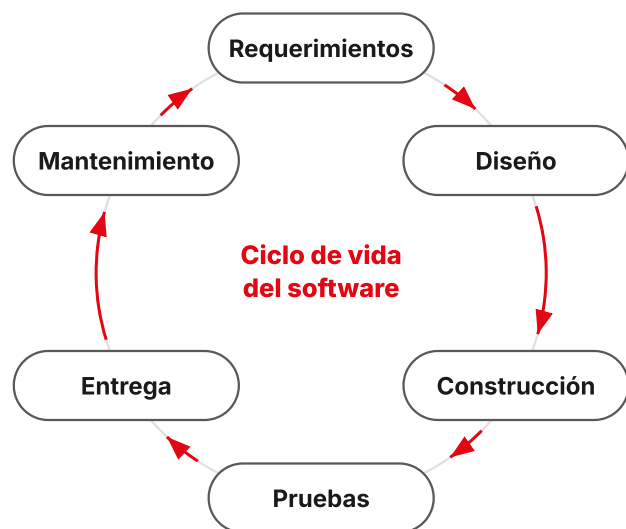
De dónde viene · 1968

Se consolidó en una conferencia de la OTAN en Garmisch, Alemania, ante la **crisis del software**: proyectos atrasados, más caros de lo previsto y con fallas.

	PROGRAMAR	INGENIERÍA DE SOFTWARE
Objetivo	Que el código funcione	Que resuelva el problema del cliente
Horizonte	Hoy	Toda la vida del sistema
Trabajo	Individual	En equipo, con roles
Evidencia	«En mi computador funciona»	Requisitos, pruebas y documentación
Restricciones	No se miden	Plazo, costo y calidad, medidos

INGENIERÍA DE SOFTWARE

Las etapas del ciclo de vida, y dónde se trabaja cada una



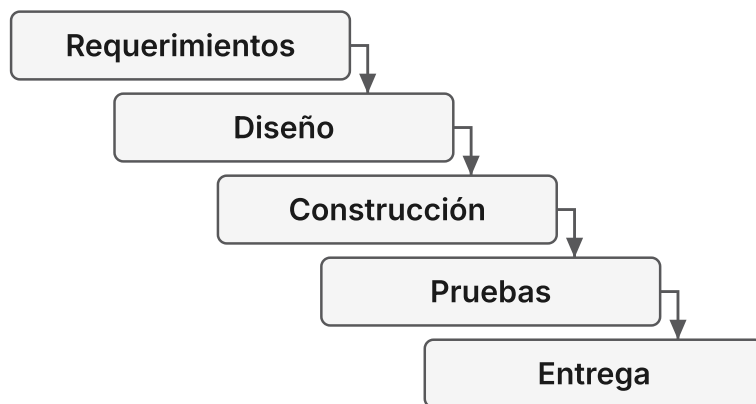
ETAPA	QUÉ PRODUCE	BLOQUE
Requerimientos	El catálogo priorizado	03 · 04
Diseño	Casos de uso, clases y BPMN	05 a 07
Construcción	Código persistente y seguro	08
Pruebas	Verificación contra los criterios de aceptación	04 · 11
Entrega	Repositorio e informe técnico	10 · 11
Mantenimiento	Que otro equipo pueda continuar el sistema	11

Las etapas son las mismas en todo método. Lo que cambia es **cómo se recorren en el tiempo**: una vez, en cascada, o muchas veces, en ciclos cortos.

CONTEXTO

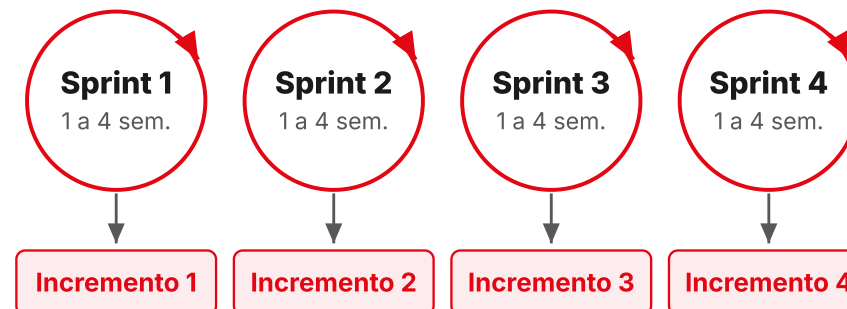
Por qué la industria del software cambió de método

CASCADA • un solo recorrido



El cliente ve el producto recién al final

ÁGIL • ciclos cortos que se repiten



Algo que funciona al cierre de cada sprint

En software, **los requerimientos cambian mientras se construye**: el cliente descubre lo que necesita al ver el producto funcionando. Un método que solo muestra el resultado al final descubre los errores cuando corregirlos es más caro.

CONCEPTO

Por qué se trabaja por iteraciones



De una sola vez

Se planifica todo al principio y se entrega todo al final. El primer momento en que alguien ve algo funcionando es el último día.

Si algo se entendió mal, se descubre cuando ya no hay tiempo de corregirlo.



Por iteraciones

Se divide en períodos cortos y fijos. Al cierre de cada uno hay **algo que funciona** y que se puede mostrar, aunque sea poco.

El error se descubre al final de la primera iteración, cuando todavía hay margen.

Una iteración **no** es una etapa. No existe la iteración «de análisis» y la iteración «de programación»: en cada una se analiza, se construye y se prueba una parte completa del sistema.

EL ORIGEN • 2001

El Manifiesto Ágil: cuatro valores

SE VALORA MÁS		QUE
Individuos e interacciones	por sobre	procesos y herramientas
Software funcionando	por sobre	documentación extensiva
Colaboración con el cliente	por sobre	negociación contractual
Respuesta ante el cambio	por sobre	seguir un plan

Quiénes y cuándo

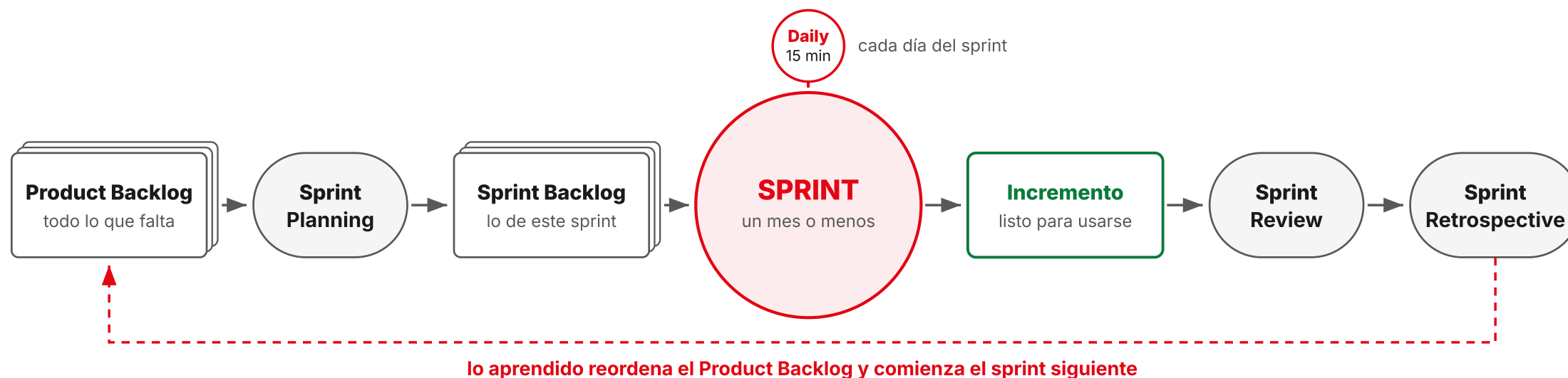
Diecisiete desarrolladores reunidos en 2001 lo firmaron. Entre ellos, Kent Beck, Martin Fowler, Ken Schwaber y Jeff Sutherland, los dos últimos autores de Scrum.

Lo que no dice: que la documentación o el plan sobren. Su frase final: «aunque valoramos los elementos de la derecha, valoramos más los de la izquierda».

Fuente: Manifiesto por el Desarrollo Ágil de Software, versión en español · agilemanifesto.org/iso/es/manifesto.html

SCRUM

Scrum en un esquema: el ciclo completo



El nombre viene del *scrum* del rugby: la formación en que el equipo avanza junto. Lo propusieron Takeuchi y Nonaka en *Harvard Business Review* (1986), y Schwaber y Sutherland lo llevaron al software. Esquema basado en la Guía Scrum 2020.

SCRUM • EL EQUIPO

Tres responsabilidades en un solo equipo

🎯 Product Owner

Dueño del producto

Responde por **el valor** de lo que se construye. Define el objetivo del producto y **ordena el Product Backlog**: decide qué va primero. Es una persona, no un comité.



Scrum Master

Facilitador del marco Scrum

Responde por **la efectividad del equipo**. Establece Scrum como la guía lo define, facilita los eventos y remueve lo que impide avanzar.

<> Developers

Quienes construyen

Responden por **el incremento** de cada sprint. Planifican el Sprint Backlog, deciden **cómo** se hace el trabajo y cumplen la definición de terminado.

Diez personas o menos. Los equipos pequeños se comunican mejor y producen más.

Sin jefes ni subequipos. Nadie manda sobre los demás: cada uno responde por algo distinto.

Fuente: Schwaber y Sutherland, *La Guía Scrum*, 2020 • scrumguides.org

SCRUM • EL ROL QUE MÁS SE CONFUNDE

El Scrum Master facilita; no dirige

A QUIÉN SIRVE	CÓMO LO HACE
Al Product Owner	Le ayuda a definir el objetivo del producto y a mantener ordenado el Product Backlog
A los Developers	Los orienta a autogestionarse, remueve impedimentos y cuida que los eventos sean productivos
A la organización	Enseña cómo funciona Scrum y elimina barreras entre el equipo y los interesados

NO ES

- El jefe del equipo
- Quien reparte las tareas
- Quien programa lo difícil
- El secretario de las reuniones

La guía los llama **«líderes que sirven»**. Si algo bloquea al equipo, no lo resuelve él: consigue que quien corresponde lo resuelva.

Fuente: *La Guía Scrum*, 2020, sección «Scrum Master» • [scrumguides.org](https://www.scrumguides.org)

SCRUM · LOS EVENTOS

Cinco eventos, cada uno con un tiempo máximo

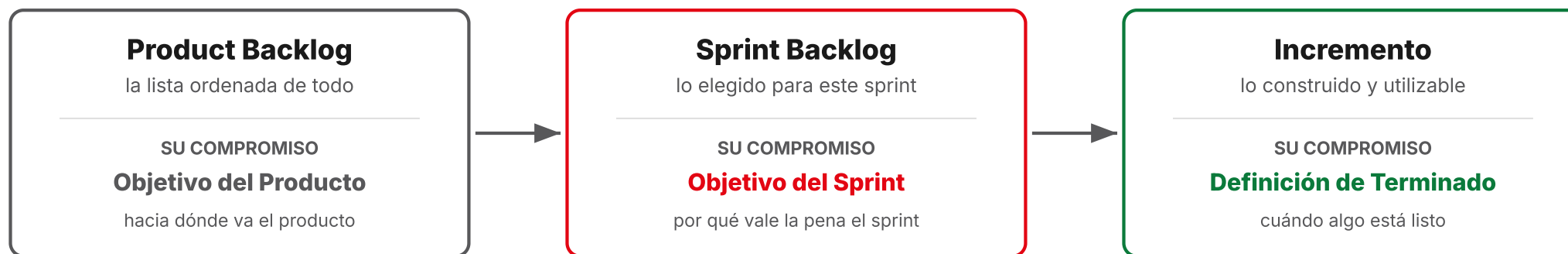
EVENTO	PARA QUÉ SIRVE	DURACIÓN MÁXIMA
Sprint	Contiene a todos los demás. Al terminar, hay un incremento utilizable	Un mes o menos
Sprint Planning	Decidir por qué vale la pena el sprint, qué se hará y cómo	8 horas
Daily Scrum	Revisar el avance hacia el objetivo del sprint y ajustar el plan del día	15 minutos, cada día
Sprint Review	Mostrar el incremento a los interesados y decidir qué sigue	4 horas
Sprint Retrospective	Revisar cómo trabajó el equipo y acordar qué mejorar	3 horas

Duraciones para un sprint de un mes; con sprints más cortos suelen ser más breves. La Review mira **el producto**; la Retrospective, **al equipo**.

Fuente: *La Guía Scrum*, 2020, sección «Eventos de Scrum» · [scrumguides.org](https://www.scrumguides.org)

SCRUM • LOS ARTEFACTOS

Tres artefactos, cada uno con su compromiso



☰ Ejemplo • definición de terminado

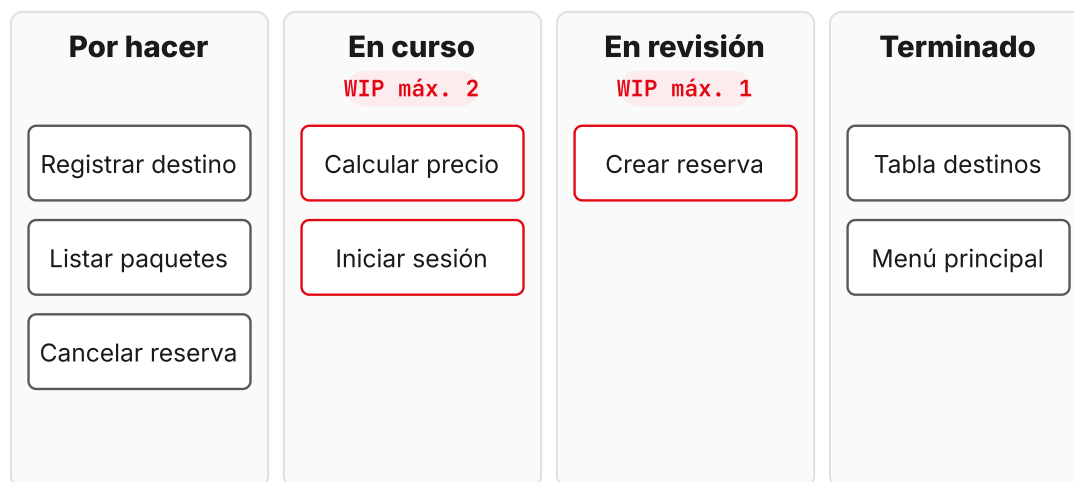
En Viajes Aventura, una funcionalidad está **terminada** cuando se ejecuta desde el menú, maneja sus errores, usa consultas parametrizadas, otro integrante la probó y está en el repositorio con su commit.

Sin definición de terminado, «está casi listo» no significa nada. Con ella, **todo el equipo mide con la misma vara.**

Fuente: *La Guía Scrum*, 2020, sección «Artefactos de Scrum» • scrumguides.org

EL OTRO ENFOQUE ÁGIL

Kanban: el flujo continuo y el límite de trabajo en curso



WIP (trabajo en curso, del inglés *work in progress*): cuántas tareas caben a la vez en una columna. Si «En curso» está llena, **nadie empieza algo nuevo**: se ayuda a terminar.

	SCRUM	KANBAN
Ritmo	Sprints de duración fija	Flujo continuo, sin iteraciones
Roles	Tres responsabilidades definidas	No define roles
Cambios	Se planifican al iniciar cada sprint	Entran cuando se libera capacidad
Se usa en	Desarrollo de producto	Soporte, operaciones, mantención

Fuente: Coleman y Vacanti, *The Kanban Guide*, 2025 · kanbanguides.org. Muchos equipos combinan ambos.

LA INDUSTRIA

Por qué las empresas de tecnología trabajan así



Valor temprano

El cliente usa una primera versión en semanas, no al final del proyecto. Lo que ya funciona empieza a rendir antes.



Riesgo acotado

Un error de comprensión se descubre al cierre del primer sprint, cuando cuesta días corregirlo y no meses.



Cambio incorporado

Los productos digitales cambian con el mercado. El Product Backlog se reordena en cada sprint sin rehacer el plan completo.



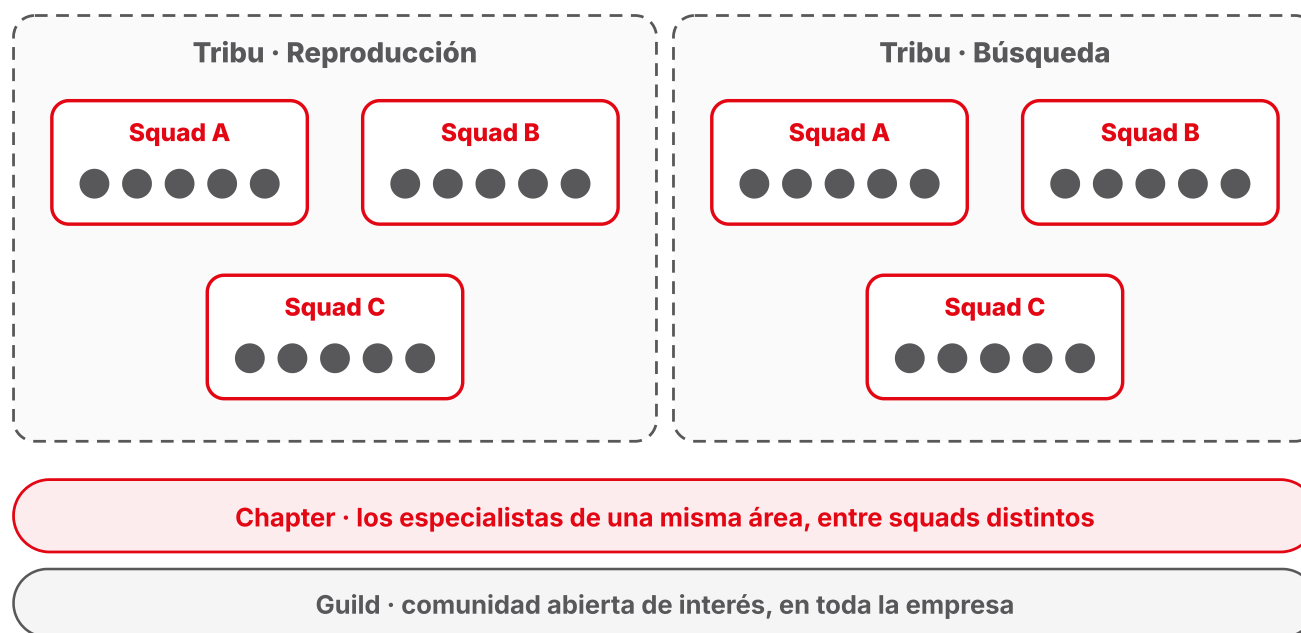
Avance visible

Lo que se muestra en la Sprint Review funciona. Es una medida más honesta que el porcentaje de un plan.

Por eso aparece en las ofertas de trabajo. Un programador que ingresa a un equipo de desarrollo participará desde el primer día en un Daily Scrum, en un Sprint Planning y en un tablero.

UN CASO DE LA INDUSTRIA

Spotify: equipos pequeños y autónomos, a escala



Squad

Equipo pequeño y autónomo, dueño de una parte del producto. Funciona como un equipo Scrum. Varios squads de un área forman una **tribu**.

Advertencia: describe a Spotify en 2012. Se replica el principio, no el organigrama.

Fuente: Kniberg e Ivarsson, *Scaling Agile @ Spotify with Tribes, Squads, Chapters & Guilds*, 2012 • blog.crisp.se

PARA PROFUNDIZAR

Fuentes oficiales, todas de acceso libre

**La Guía Scrum 2020, en español latinoamericano**

scrumguides.org • [Guía Scrum en español](#)

El documento oficial: 14 páginas. Todo lo que es Scrum está ahí, y lo que no está ahí no es Scrum.

**Manifiesto por el Desarrollo Ágil de Software**

agilemanifesto.org/iso/es/manifesto.html

Los cuatro valores. Los doce principios están enlazados en la misma página.

**The Kanban Guide**

kanbanguides.org

Las tres prácticas de Kanban y el límite de trabajo en curso. Disponible en varios idiomas.

**The New New Product Development Game • Takeuchi y Nonaka, 1986**

hbr.org/1986/01/the-new-new-product-development-game

El artículo de Harvard Business Review del que viene el nombre Scrum.

**Scaling Agile @ Spotify • Kniberg e Ivarsson, 2012**

blog.crisp.se/2012/11/14/henrikkniberg/scaling-agile-at-spotify

Cómo una empresa organizó decenas de equipos ágiles. Se lee como un caso, no como una receta.

BLOQUE

02

Una unidad que **integra las anteriores**

Reúne el trabajo completo: todo lo que ya sabe hacer, aplicado a un solo sistema y documentado como corresponde.

EL PUNTO DE PARTIDA

Qué aporta cada unidad, y qué agrega esta

UNIDAD	LO QUE YA SABE HACER	DÓNDE SE USA AHORA
1 • Modelado	Identificar entidades, escribir clases con atributos y métodos, y representar relaciones y multiplicidad en UML	El diagrama de clases del sistema
2 • Python y datos	Llevar el diagrama a clases, conectar con SQLite mediante librería oficial, ejecutar el CRUD y controlar errores con try-except	La aplicación que se entrega
3 • Seguridad	Autenticar con credenciales cifradas, validar entradas, proteger datos sensibles y no exponer información en los errores	El acceso al sistema
4 • Esta unidad	Agrega cinco cosas: levantar requerimientos, diagramar casos de uso, modelar el proceso en BPMN, planificar con metodologías ágiles y escribir el informe técnico	

Nada de lo anterior se explica desde cero. Lo que cambia es que **todo tiene que funcionar junto** y quedar explicado por escrito.

EL RECORRIDO

De lo que el cliente dice al sistema que funciona

#	ETAPA	LA PREGUNTA QUE RESPONDE	LO QUE DEJA POR ESCRITO
1	Requerimientos	¿Qué pidió exactamente el cliente?	Catálogo numerado y priorizado
2	Casos de uso	¿Quién usa el sistema y para qué?	Diagrama de casos de uso y sus fichas
3	Clases	¿De qué piezas está hecho?	Diagrama de clases en UML
4	Proceso	¿En qué orden ocurre el trabajo?	Diagrama BPMN del proceso principal
5	Planificación	¿Quién hace qué y para cuándo?	Roles, backlog e iteraciones
6	Construcción	¿Funciona, persiste y es seguro?	El código y la base de datos
7	Documentación	¿Por qué está hecho así?	El informe técnico

Cada etapa consume lo que produjo la anterior. Una clase que no proviene de un caso de uso, y un caso de uso que no proviene de un requerimiento, **no se pueden justificar**, porque nada los pidió.

BLOQUE

03

El caso:

Viajes Aventura

Una agencia que arma paquetes combinando destinos, y que hoy administra todo en una planilla y un cuaderno.

EL PUNTO DE PARTIDA

El cliente ya dijo lo que necesita

«La empresa necesita implementar un sistema de software que permita gestionar su operación, integrando la administración de **destinos, paquetes turísticos y reservas de clientes.**»

Del encargo del cliente



Y también dijo qué le duele

- No hay control de los destinos disponibles
- La gestión de paquetes es difícil de sostener
- Falla el registro y seguimiento de reservas
- Los datos sensibles de clientes están en riesgo
- Todo es manual, y eso afecta la eficiencia

Esas dos listas **son** la materia prima. El trabajo no es inventar qué debería hacer el sistema: es **traducir lo que el cliente declaró** a requerimientos numerados que se puedan construir y comprobar.

LO QUE EL ENCARGO DECLARA

Cuatro conjuntos de necesidades, un solo sistema



Gestión de destinos

Registrar destinos nuevos · modificar su información · eliminar destinos existentes · listar los disponibles.



Gestión de paquetes turísticos

Crear paquetes combinando varios destinos · definir fechas y costos · consultar disponibilidad · calcular el costo según los destinos elegidos.



Gestión de reservas

Registrarse y autenticarse · reservar paquetes · almacenar las reservas en la base de datos · consultar el historial.



Gestión del sistema

Autenticación de usuarios · validación de los datos ingresados · protección de credenciales y datos sensibles.

Es un solo sistema, no cuatro programas. Estos son los conjuntos en que el encargo ordena sus necesidades, y por eso conviene agrupar también el catálogo de requerimientos.

EL DOCUMENTO DE ANTECEDENTES

Qué trae, sección por sección

#	SECCIÓN	PARA QUÉ SIRVE EN EL TRABAJO
1	El negocio	El catálogo de destinos con sus costos, los paquetes con sus cupos y los dos roles que usan el sistema
2	Cómo trabajan hoy	La planilla real, con reservas duplicadas y un mismo paquete cobrado a dos precios. Cada defecto anuncia una validación
3	Entrevistas	Tres socios, tres problemas distintos. De aquí salen los requerimientos y se distingue lo urgente de lo deseable
4	La temporada en cifras	Siete indicadores que permiten priorizar con evidencia , y no por gusto
5	Reglas de negocio	Diecisiete reglas numeradas , de R1 a R17. Son la materia prima del modelo y de las validaciones
6	Alcance y vacíos	Qué entra, qué queda fuera y qué el caso deliberadamente no responde

El documento entrega **antecedentes**. Los entregables y las condiciones de entrega los fija la guía oficial de la evaluación.

CÓMO SE USA

Dónde está cada cosa que hay que producir

LO QUE HAY QUE PRODUCIR	DE DÓNDE SALE, EN EL DOCUMENTO
Requerimientos funcionales	Los verbos del encargo y lo que piden las tres entrevistas
Requerimientos no funcionales	Los problemas declarados: datos sensibles en una planilla que circula por correo, y todo escrito a mano
Prioridad de cada requerimiento	La temporada en cifras: seis reservas sobre el cupo pesan más que cuatro diferencias de precio
Actores del diagrama de casos de uso	La sección 1.3: cliente y administrador
Clases, atributos y relaciones	Las reglas R1 a R13
Validaciones del código	Las reglas R2, R5, R14, R15 y R16
Medidas de acceso seguro	Las reglas R9, R10, R11 y R17

LA DISTINCIÓN QUE MÁS CUESTA

Una regla de negocio no es un requerimiento



Regla de negocio

Describe **cómo funciona la agencia**. Existe aunque no haya sistema.

R14 • «El cupo disponible de un paquete es su cupo máximo menos las personas ya reservadas.»



Requerimiento

Describe **qué debe hacer el sistema**, y se puede comprobar si está o no está.

RF-14 • «El sistema debe rechazar una reserva cuya cantidad de personas supere el cupo disponible del paquete, informando el motivo.»

Las diecisiete reglas **no se copian al informe como requerimientos**. Se traducen, y esa traducción es parte de lo que se evalúa. Una misma regla puede producir un requerimiento funcional, una validación y un atributo del diagrama.

LO QUE EL CASO NO RESPONDE

Cómo se declara un supuesto



Tres vacíos declarados

- Qué ocurre cuando un cliente **desiste** de una reserva
- Cómo se comporta un paquete cuando su **temporada termina**
- **Quién** puede modificar el catálogo de destinos

Y habrá otros que el equipo detecte. Ningún caso real entrega toda la información.



Un supuesto bien declarado

Tiene tres partes: **qué se supuso, por qué es razonable y qué cambiaría si fuera falso.**

«Se supone que una reserva no puede anularse, porque el caso no lo menciona y el alcance no incluye devoluciones. Si el cliente lo exigiera, habría que agregar un estado a la reserva y liberar el cupo comprometido.»

Un supuesto sin declarar es un error. Un supuesto declarado y fundamentado es criterio profesional, y así lo lee quien corrige.

BLOQUE

04

Requerimientos: del encargo al catálogo

No se inventan: se derivan de lo que el cliente declaró, y esa derivación es exactamente lo que se evalúa.

LA TÉCNICA

Del enunciado del cliente al requerimiento numerado

#	QUÉ SE HACE	POR QUÉ, Y CÓMO SE RECONOCE
1	Subrayar los verbos	Cada verbo del encargo que describe algo que el sistema debe hacer es un requerimiento funcional candidato: <i>registrar, modificar, listar, combinar, calcular, autenticar, almacenar</i>
2	Separar lo que viene junto	«Registrar y autenticar usuarios» son dos capacidades distintas: se construyen, se prueban y se evalúan por separado
3	Convertir cada dolor en un no funcional	«Los datos sensibles están en riesgo» no dice qué hace el sistema, sino cómo debe comportarse
4	Numerar y agrupar	RF-01 en adelante para los funcionales, RNF-01 para los no funcionales, agrupados por el conjunto del encargo al que pertenecen

Ningún requerimiento aparece sin que alguien lo haya pedido. Si el equipo agrega una condición propia, **la justifica en el informe.**

CONCEPTO

Funcionales y no funcionales



Funcional · qué hace

Una acción que el sistema ejecuta y que el usuario puede observar. Sale de un **verbo** del encargo.

- RF-01 · Registrar un destino turístico nuevo
- RF-05 · Crear un paquete combinando varios destinos
- RF-09 · Reservar un paquete turístico



No funcional · cómo debe comportarse

Una condición de calidad exigida a todo el sistema. Sale de un **riesgo o dolor** declarado por el cliente.

- RNF-01 · Las contraseñas se almacenan cifradas
- RNF-02 · Solo un usuario autenticado accede a las reservas
- RNF-03 · Ningún mensaje de error revela datos del sistema

Regla práctica: si se puede escribir «el sistema **debe permitir...**», es funcional. Si se escribe «el sistema **debe ser...**», es no funcional.

EL ATRIBUTO QUE DECIDE

Un requerimiento que se puede comprobar

Sin comprobar: «El sistema debe gestionar bien los paquetes turísticos.» No hay forma de decir si se cumplió.

Comprobable: «El sistema debe calcular el costo de un paquete como la suma del costo de los destinos que lo componen, y mostrarlo antes de confirmar la reserva.» Se ejecuta y se responde sí o no.

✓ Las cuatro condiciones

Necesario

Responde a algo que el cliente declaró.

Sin ambigüedad

Se lee de una sola manera.

Verificable

Existe una prueba que responde si se cumple.

Independiente

Es uno solo. Si tiene un «y», son dos.

EJEMPLO RESUELTO

Gestión de destinos, derivada paso a paso

LO QUE DICE EL ENCARGO	CÓDIGO	EL REQUERIMIENTO, REDACTADO	PRIORIDAD
«Registrar nuevos destinos turísticos»	RF-01	El sistema debe permitir registrar un destino con nombre, país, descripción y costo base	MUST
«Modificar información de destinos»	RF-02	El sistema debe permitir modificar los datos de un destino ya registrado	SHOULD
«Eliminar destinos existentes»	RF-03	El sistema debe permitir eliminar un destino que no forme parte de ningún paquete vigente	SHOULD
«Listar destinos disponibles»	RF-04	El sistema debe permitir listar los destinos registrados, con su país y su costo base	MUST

Note el RF-03: el encargo decía solo «eliminar destinos». La condición «que no forme parte de ningún paquete vigente» la **agregó el equipo**, y esa decisión se justifica en el informe. Ahí empieza el criterio técnico.

LA FORMA DE ESCRIBIRLO

Historia de usuario y criterios de aceptación

HISTORIA DE USUARIO

Como <rol>, **quiero** <acción>, **para** <beneficio>.

Como cliente, quiero reservar un paquete turístico, para asegurar mi cupo antes de que se agote.

La tercera parte —el «para»— es la que permite priorizar: sin ella no se sabe qué se pierde si el requerimiento no se construye.



Criterios de aceptación

Las condiciones que deben cumplirse para dar la historia por terminada. Se escriben **antes** de programar.

- Solo un usuario autenticado puede reservar
- No se reserva un paquete fuera de sus fechas
- No se reserva por sobre el cupo disponible
- La reserva queda almacenada con su fecha y su costo
- El sistema explica con un mensaje claro si algo lo impide

CÓMO SE ORDENA EL CATÁLOGO

[VA AL INFORME • 4.1.1](#)

Agrupar, numerar y priorizar con MoSCoW

El código es la identidad

El número no cambia nunca, aunque el requerimiento se reescriba. Es lo que permite citarlo en el caso de uso, en el tablero, en el commit y en la matriz de trazabilidad del informe.

Se agrupa por el conjunto del encargo: destinos, paquetes, reservas y sistema. Un catálogo de veinte requerimientos sin agrupar no se puede revisar.

☰ Priorizar con MoSCoW

Must

Sin esto el sistema no sirve. Va en la primera iteración

Should

Se espera, pero el sistema funciona sin ello

Could

Se construye si sobra tiempo

Won't

Se declara por escrito que no se hará, y por qué

La última fila es la que más protege: **declarar lo que queda fuera** evita que se reclame después.

Casos de uso: quién usa el sistema y para qué

El puente entre el catálogo de requerimientos y el diagrama de clases. Es lo único de esta etapa que no se vio antes.

CONCEPTO

Qué pregunta responde, y quién es un actor

CASO DE USO

Un **objetivo completo** que un actor logra usando el sistema. Empieza cuando el actor lo inicia y termina cuando obtiene un resultado que le sirve.

© La prueba para reconocerlo

Si el actor pudiera irse satisfecho después de ejecutarlo, es un caso de uso. «Reservar un paquete» lo es. «Validar la fecha» no: nadie entra al sistema a validar una fecha.

**El actor es un rol, no una persona**

Una misma persona puede ser dos actores, y un actor puede no ser una persona.

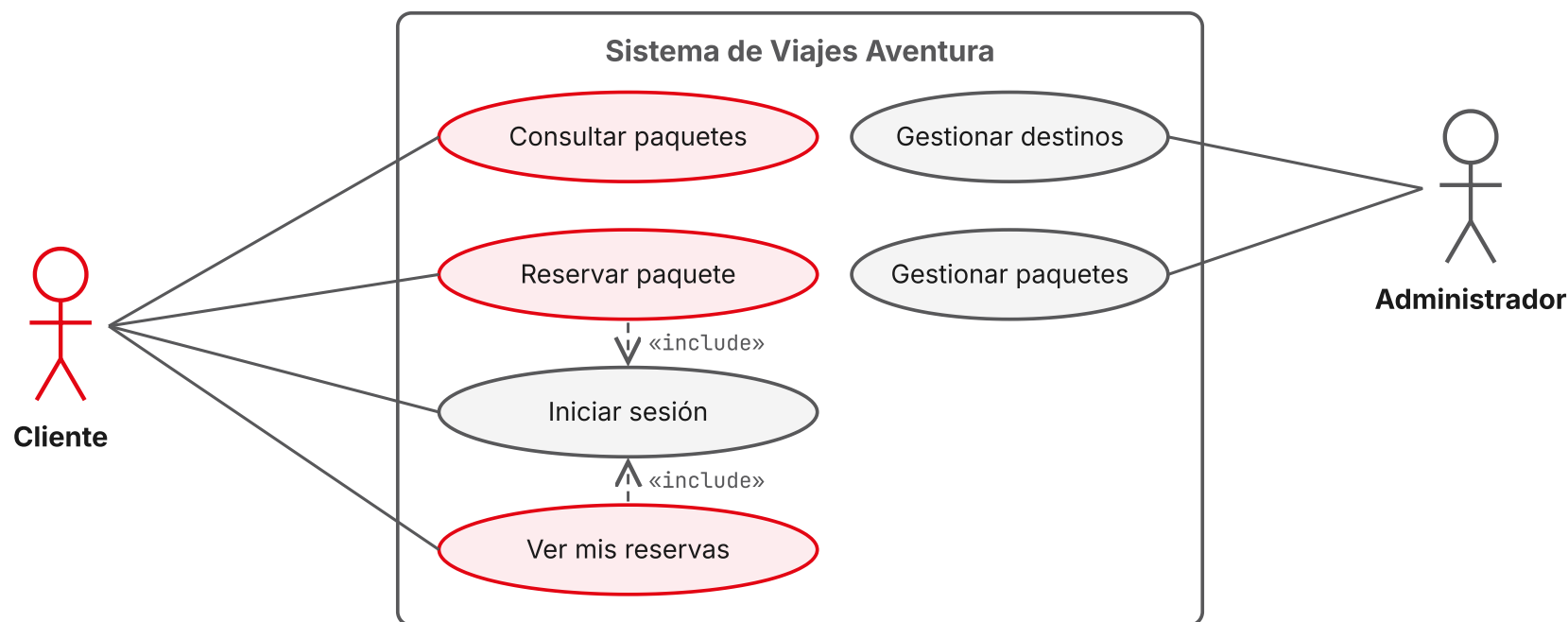
- **Cliente** · consulta paquetes y reserva
- **Administrador** · mantiene destinos y paquetes

Los dos actores salen del encargo: hay quien *usa* la agencia y quien la *administra*.

EL DIAGRAMA

[VA AL INFORME • 4.1.2](#)

Los casos de uso del sistema, con sus dos actores



Actor: la figura, siempre fuera del recuadro · **Caso de uso:** la elipse, nombrada con un verbo · **Límite del sistema:** el recuadro, lo de adentro se construye · **«include»:** lo que ese caso siempre necesita.

EL DETALLE QUE NO CABE EN EL DIBUJO

 VA AL INFORME • 4.1.2

La ficha de «Reservar paquete»

Nombre	Reservar paquete
Actor principal	Cliente
Requerimientos	RF-09, RNF-02
Precondición	El cliente inició sesión
Postcondición	La reserva queda almacenada con su fecha y costo

Flujo principal

1. El sistema muestra los paquetes disponibles
2. El cliente selecciona uno y ve su costo
3. El cliente confirma la reserva
4. El sistema la almacena y muestra el comprobante

Flujo alternativo · el que suele faltar

- 2a.** El paquete está fuera de sus fechas: el sistema lo informa y vuelve al listado.

LO QUE SE CORRIGE

Cuatro errores frecuentes en casos de uso

Confundir caso de uso con paso. «Ingresar la contraseña» no es un caso de uso: es un paso dentro de «Iniciar sesión».

Poner al actor dentro del recuadro. El actor es externo por definición; si está adentro, es una clase.

Una elipse por operación CRUD. Crear, leer, actualizar y eliminar no son cuatro objetivos: son las cuatro caras de «Gestionar destinos».

Nombrar con sustantivo. «Gestión de reservas» no dice qué logra nadie. Corresponde «Reservar paquete».

BLOQUE

06

El diagrama de clases: el modelado de la Unidad 1

El mismo método de la primera unidad, aplicado ahora a un sistema que además debe construirse y sostenerse.

EL MÉTODO, RECORDADO

Del caso de uso a la clase, y de la clase al método

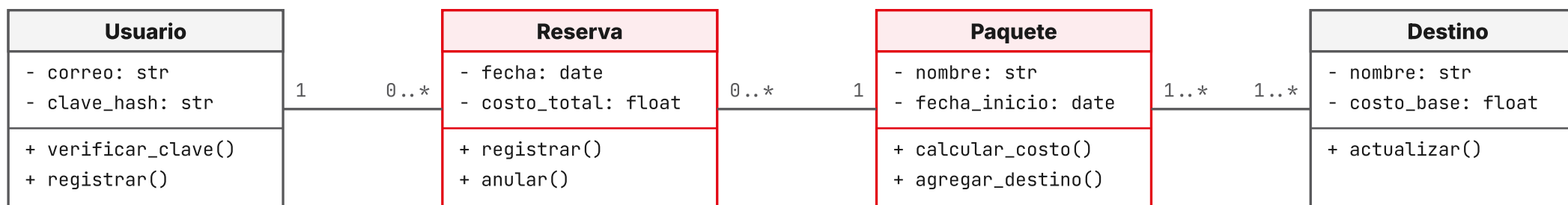
REQUERIMIENTO	CASO DE USO	CLASE QUE APARECE	MÉTODO QUE APARECE
RF-01 Registrar destino	Gestionar destinos	Destino, DestinoRepositorio	guardar()
RF-08 Calcular el costo del paquete	Consultar paquetes	Paquete	calcular_costo()
RF-09 Reservar un paquete	Reservar paquete	Reserva, ReservaRepositorio	registrar()
RNF-01 Contraseñas cifradas	Iniciar sesión	Usuario	verificar_clave()

Es la técnica de la unidad 1: los **sustantivos** del caso de uso dan las clases candidatas, y los **verbos**, los métodos. Lo nuevo es que ahora esa cadena se escribe completa en el informe.

EL RESULTADO

[VA AL INFORME • 4.1.2](#)

El diagrama de clases del sistema



Un usuario tiene muchas reservas • una reserva es de un paquete • un paquete combina varios destinos, y un destino está en varios paquetes

La relación de muchos a muchos entre Paquete y Destino es la decisión que define este sistema

LA DECISIÓN DE MODELADO

Qué hacer con una relación de muchos a muchos

🔗 El problema

Un paquete combina varios destinos y un destino puede estar en varios paquetes. Una base de datos relacional **no puede guardar eso en dos tablas**: no hay dónde poner la clave.

📦 La solución

Una **tercera tabla**, paquete_destino, con la clave del paquete y la del destino. Cada fila indica que ese destino integra ese paquete.

```
CREATE TABLE paquete_destino (  
    paquete_id INTEGER NOT NULL,  
    destino_id INTEGER NOT NULL,  
    PRIMARY KEY (paquete_id, destino_id),  
    FOREIGN KEY (paquete_id) REFERENCES paquete(id),  
    FOREIGN KEY (destino_id) REFERENCES destino(id));
```

Es una decisión técnica que corresponde poder fundamentar: **por qué tres tablas y no dos**, y cómo se recorre la intermedia para calcular el precio del paquete.

BPMN:

el orden en que ocurre el trabajo

El diagrama de clases describe de qué está compuesto el sistema; BPMN, el orden en que ocurre el trabajo y quién ejecuta cada paso.

DOS DIAGRAMAS, DOS PREGUNTAS

Qué responde cada notación

Diagrama de clases · UML 2.5.1

De qué está hecho el sistema. Es una fotografía de la estructura: no tiene tiempo ni orden.

Ya lo sabe hacer desde la unidad 1.

Diagrama de procesos · BPMN 2.0

Cómo ocurre el trabajo. Es una película: tiene principio, orden, bifurcaciones y final.

Es nuevo en esta unidad.

BPMN

Business Process Model and Notation · Notación y Modelo de Procesos de Negocio, norma **ISO/IEC 19510**, versión 2.0. Es un estándar, no una convención: los símbolos significan lo mismo en cualquier organización del mundo.

NOTACIÓN

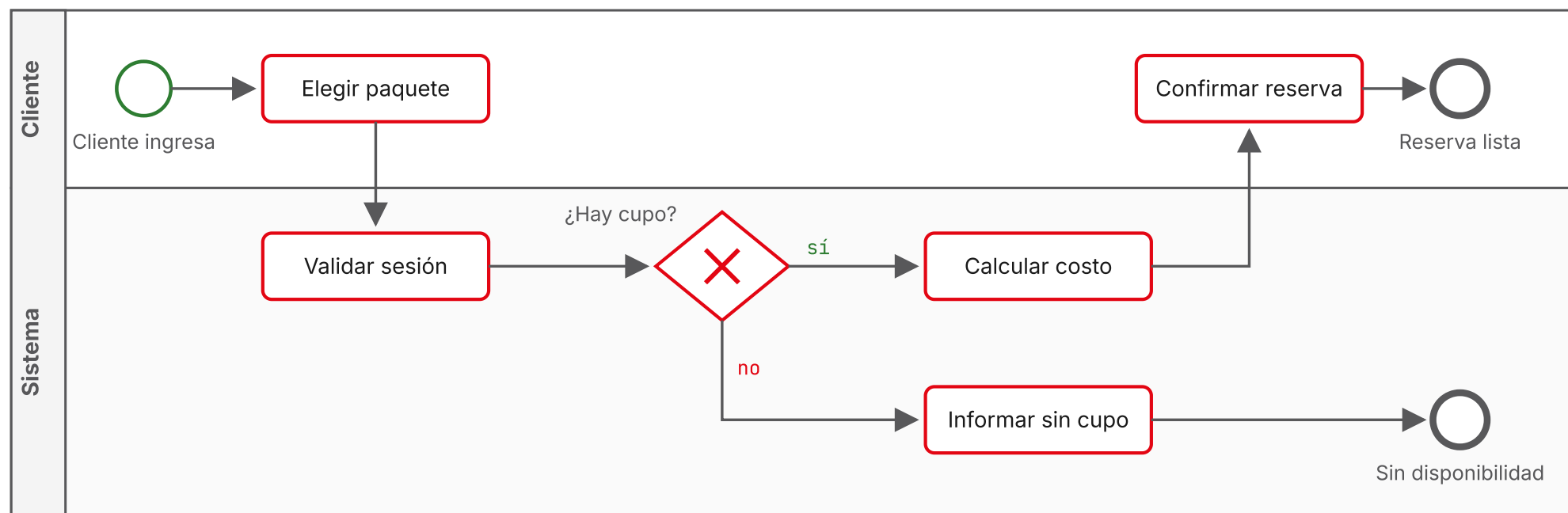
Los cinco elementos que hacen falta

SÍMBOLO	NOMBRE	QUÉ REPRESENTA	CÓMO SE NOMBRA
	Evento de inicio	Lo que dispara el proceso. Uno solo por diagrama	Con un sustantivo: «Solicitud recibida»
	Evento de fin	Dónde termina cada camino. Puede haber varios	«Reserva registrada», «Solicitud rechazada»
	Tarea	Una unidad de trabajo que alguien ejecuta	Verbo + objeto: «Verificar disponibilidad»
	Compuerta	Donde el flujo se abre en caminos excluyentes	Con una pregunta: «¿Hay cupo?»
	Flujo de secuencia	El orden. Nunca se cruza ni queda suelto	Sin rótulo, salvo al salir de una compuerta

EJEMPLO COMPLETO

VA AL INFORME • 4.1.2

El proceso de reservar, con sus dos carriles



Los **carriles** distinguen un BPMN de un diagrama de flujo: cada tarea está en el carril de quien la ejecuta. La compuerta abre **dos caminos excluyentes**, y cada uno llega a su propio evento de fin.

LO QUE SE CORRIGE

Los cuatro errores de BPMN

Dibujar el código, no el proceso. Si aparece «recorrer la lista» o «abrir la conexión», se está dibujando un algoritmo. BPMN describe trabajo del negocio.

Una compuerta sin pregunta. El rombo sin rótulo y sin «sí/no» en sus salidas no comunica la decisión.

Sin carriles. Sin ellos no se sabe quién ejecuta cada tarea, que es la mitad de lo que aporta la notación.

Caminos que no terminan. Toda rama que sale de una compuerta debe llegar a un evento de fin.

Del modelo al código: **persistencia y acceso seguro**

Nada nuevo que aprender: la persistencia de la Unidad 2 y la seguridad de la Unidad 3, sobre un sistema de cuatro entidades.

UNIDAD 2, RECORDADA

El CRUD y el control de errores

Consulta parametrizada: el dato viaja aparte, nunca concatenado

```
def guardar(self, destino):  
    with sqlite3.connect(self.ruta) as con:  
        con.execute("INSERT INTO destino (nombre, pais, costo_base) "  
                    "VALUES (?, ?, ?)",  
                    (destino.nombre, destino.pais, destino.costo_base))
```

Lo que ya sabe: las cuatro operaciones, la conexión con librería oficial y try-except para ValueError e IntegrityError.

Lo que se sigue corrigiendo: armar la consulta con f"...{'{' }dato{'{' } }...". Abre la puerta a la inyección de SQL y cuesta el criterio de seguridad completo.

UNIDAD 3, RECORDADA

Autenticación, validación y datos sensibles



Autenticación

Registro e inicio de sesión con la contraseña **cifrada**, mediante una librería oficial. En la base nunca se guarda el texto original.



Validación de entradas

Todo dato que llega del usuario se revisa antes de usarse: que no venga vacío, que sea del tipo esperado y que esté en el rango razonable.



Datos sensibles

Las credenciales viven **fuera del código**, en variables de entorno. Y ningún mensaje de error muestra rutas, consultas ni datos de otros usuarios.

Lo que cambia respecto de la unidad 3: allá la seguridad se aplicaba a un script. Aquí protege un sistema con reservas de personas reales, y hay que poder explicar **de qué** protege cada medida, no solo que está puesta.

El ciclo, y la tabla que va al informe



Pedir, leer, corregir, registrar

El mismo ciclo de la unidad 2. Lo que cambia es que ahora **queda por escrito**: cada fragmento con su decisión y su fundamento.

«El uso de código generado exclusivamente por IA, sin análisis ni ajustes, será considerada insuficiente.»

QUÉ SE PIDIÓ	DECISIÓN	FUNDAMENTO TÉCNICO
Consulta para buscar paquetes por destino	DESCARTADO	Armaba la consulta concatenando el texto ingresado: permite inyección de SQL
Función para cifrar la contraseña	MODIFICADO	Usaba un algoritmo sin sal. Se reemplazó por la librería oficial
Cálculo del costo del paquete	ADOPTADO	Recorre la tabla intermedia correctamente. Se probó con tres paquetes

La última columna es la que vale. Las dos primeras describen; la tercera demuestra criterio técnico.

BLOQUE

09

Planificación ágil: roles, backlog e iteraciones

Scrum aplicado a un equipo de tres: quién coordina, qué lista ordena el trabajo y cómo se planifica cada iteración.

APLICADO AL PROYECTO

Scrum en un equipo de tres: cómo se adapta

ELEMENTO DE SCRUM	EN EL PROYECTO VIAJES AVENTURA
Product Owner	El cliente. Sus necesidades están en el documento de antecedentes, y el docente responde por él las consultas
Scrum Master	El líder de proyecto , rotativo: facilita las reuniones, detecta lo que bloquea y reporta el avance
Developers	Los tres integrantes. Todos programan sobre el sistema completo
Sprint	Una iteración corta que termina con algo que se ejecuta: primero destinos, luego paquetes y reservas
Daily Scrum	Quince minutos al comenzar cada sesión de trabajo: qué se avanzó, qué sigue y qué lo impide
Review y Retrospective	Mostrar lo que funciona, y acordar una sola mejora concreta para la iteración siguiente

Scrum no se aplica «entero o nada». Un equipo pequeño toma **la iteración, la lista ordenada y la revisión frecuente**, que es donde está el valor.

LOS ROLES

Reparten responsabilidad, no partes del sistema



Líder de proyecto

Coordina las actividades, gestiona las reuniones de seguimiento, supervisa el cumplimiento de tareas y es el interlocutor con el docente. El rol es **rotativo**.



Responsable de datos

Responde por el esquema de las cinco tablas, por las clases de persistencia y por que ninguna consulta se arme concatenando texto.



Responsable de seguridad

Responde por la autenticación, por la validación de entradas y por que ninguna credencial quede escrita en el código.

Nadie es dueño de una parte del sistema. Los tres programan sobre las cuatro entidades. Lo que reparte el rol es quién responde cuando algo de esa área falla. **Cada integrante debe poder explicar el sistema entero**, no solo lo que escribió.

LA LISTA QUE ORDENA TODO

 VA AL INFORME • 4.1.3

El Product Backlog

ID	HISTORIA	PRIORIDAD	ESTIM.
RF-12	Como cliente, quiero registrarme e iniciar sesión	MUST	4 h
RF-01	Como administrador, quiero registrar un destino	MUST	3 h
RF-05	Como administrador, quiero crear un paquete con varios destinos	MUST	6 h
RF-09	Como cliente, quiero reservar un paquete	MUST	5 h
RF-11	Como cliente, quiero ver el historial de mis reservas	SHOULD	3 h

PRODUCT BACKLOG

La lista **única y ordenada** de todo lo que falta por construir en el proyecto completo.

- ☰ Se ordena por **prioridad**, no por facilidad
- ↺ Es **vivo**: se reordena cuando algo cambia
- 🔗 Cada línea **cita su requerimiento**

EL COMPROMISO DE UN PERÍODO

[VA AL INFORME • 4.1.3](#)

El Sprint Backlog

SPRINT BACKLOG

Lo que el equipo **se compromete a terminar** en esta iteración: se toma de arriba del Product Backlog y se parte en tareas con responsable.

La diferencia entre los dos

El **Product Backlog** es todo el proyecto y cambia de orden. El **Sprint Backlog** es una sola iteración y **no se toca** una vez comprometido: lo nuevo que aparece espera su turno.

ITERACIÓN 1 • TAREAS	RESPONSABLE	HORAS
Crear las cinco tablas en base.py	Datos	3
Clase Usuario con contraseña cifrada	Seguridad	3
Menú de registro e inicio de sesión	Líder	2
CRUD completo de Destino	Datos	3

El tablero refleja ese compromiso en tres columnas: por hacer, en curso —una por persona— y terminado, entendiendo por terminado «probado y subido al repositorio».

BLOQUE

10

Trabajo en equipo: un repositorio, tres personas

Cómo tres personas escriben sobre el mismo código sin sobrescribirse, y cómo el historial acredita quién hizo qué.

HÁGALO AHORA

El ciclo completo de una rama

Traer lo último y abrir la rama de la tarea

```
git switch main && git pull && git switch -c feature/reservas
```

Guardar y publicar el avance, con un mensaje que explique

```
git add reserva.py && git commit -m "feat(reservas): registrar"
```

```
git push -u origin feature/reservas
```

```
git switch main && git pull && git merge feature/reservas
```

CUANDO GIT AVISA

El conflicto no es un error: es una decisión pendiente

```
<<<<<< HEAD
    if self.cupos > 0:
=====
    if self.cupos >= 1 and self.vigente:
>>>>>> feature/reservas
```

Git encontró que **dos personas cambiaron la misma línea** y no puede decidir cuál vale. Marca las dos y espera.

- 🕒 Se leen las dos versiones y se conversa cuál corresponde
- ✏ Se deja **una sola** y se borran las líneas de marca
- ✓ git add y git commit cierran el conflicto

Lo que enseña un conflicto: que dos integrantes entendieron distinto la misma regla del negocio. Resolverlo en el código sin conversarlo deja la discrepancia viva en otra parte del sistema.

BLOQUE

11

El informe técnico: **cómo se redacta**

El documento que acompaña a la aplicación: no enumera lo realizado, sino que fundamenta cada decisión y acredita su cumplimiento.

EL ENTREGABLE

Qué es un informe técnico, y para quién se escribe

INFORME TÉCNICO

El documento que permite a **otro equipo** entender el sistema, mantenerlo y continuarlo, sin hablar con quien lo escribió.

👤 Quién lo lee

Un profesional del área que **no conoce el proyecto**. Sabe programar, así que no hay que explicarle qué es una clase; no sabe nada del negocio, así que sí hay que explicarle qué es un paquete turístico en este sistema.



Las tres preguntas que debe responder

- **¿Qué se construyó?** El alcance, con lo que quedó fuera declarado
- **¿Por qué así?** Cada decisión técnica, con su fundamento y la alternativa descartada
- **¿Cómo se comprueba?** La forma de verificar que cada requerimiento se cumple

Un informe que solo responde la primera es una lista de lo hecho. **Las otras dos son las que lo vuelven técnico.**

LA ESTRUCTURA

Las secciones, una por una • 1 a 4

#	SECCIÓN	QUÉ CONTIENE	QUÉ NO VA AQUÍ
1	Problema y alcance	Lo que el cliente declaró, en sus términos, y lo que el equipo dejó fuera	Soluciones: todavía no se decide nada
2	Requerimientos	El catálogo numerado y agrupado, funcionales y no funcionales, con su prioridad	Nombres de clases ni de tablas
3	Modelado	Casos de uso, diagrama de clases y BPMN, cada uno con su lectura escrita	Diagramas sin texto que los explique
4	Planificación	Roles, Product Backlog, iteraciones y el tablero al cierre	Promesas: va lo que ocurrió, no lo planeado

Las cuatro primeras secciones se escriben **mientras se construye**, no al final: cada clase produce una.

LA ESTRUCTURA

Las secciones, una por una • 5 a 8

#	SECCIÓN	QUÉ CONTIENE	QUÉ NO VA AQUÍ
5	Implementación	Estructura del código, las cinco tablas, el acceso seguro y las decisiones que se tomaron	El código completo pegado
6	Uso de IA	La tabla de qué se pidió, qué devolvió, qué se decidió y con qué fundamento técnico	«Se usó una herramienta de apoyo»
7	Trazabilidad	La matriz que une requerimiento, caso de uso, clase, método y prueba	—
8	Conclusiones	Qué quedó pendiente y qué se haría distinto	Agradecimientos y frases de cierre

Si falta una sección, falta completo lo que esa sección sostiene, no un punto suelto.

CÓMO SE ESCRIBE

Un diagrama nunca va solo

Lo que se entrega casi siempre

Una página con el diagrama pegado y, debajo, el texto «Figura 3: diagrama de clases». Nada más.

El lector queda solo frente al dibujo, y tiene que adivinar qué se le quiso mostrar.

Lo que corresponde

Tres párrafos cortos alrededor del diagrama:

- **Antes:** qué pregunta responde este diagrama
- **Después:** la lectura de lo que el lector debe notar, nombrando los elementos
- **Después:** la decisión que refleja, y qué alternativa se descartó

Ejemplo de lectura: «La relación entre Paquete y Destino es de muchos a muchos: un paquete combina varios destinos y un destino aparece en varios paquetes. Se resolvió con la tabla intermedia paquete_destino, y no duplicando el destino en cada paquete, porque así un cambio de costo base se refleja en todos los paquetes que lo contienen.»

LO QUE AMARRA TODO

La matriz de trazabilidad

REQUERIMIENTO	CASO DE USO	CLASE	MÉTODO	CÓMO SE COMPRUEBA
RF-01 Registrar destino	Gestionar destinos	DestinoRepositorio	guardar()	Opción 1 del menú, luego listar
RF-08 Calcular el costo	Consultar paquetes	Paquete	calcular_costo()	Paquete con tres destinos de costo conocido
RF-09 Reservar paquete	Reservar paquete	ReservaRepositorio	registrar()	Opción 4 con un paquete sin cupo
RNF-01 Contraseñas cifradas	Iniciar sesión	Usuario	verificar_clave()	Consultar la tabla usuario

Demuestra en una página que **todo lo pedido está construido** y que nada se construyó sin que alguien lo pidiera. Una fila por requerimiento, sin excepción: una fila vacía señala un requerimiento sin construir.

REGISTRO

Cómo se redacta un documento técnico

COMO SE ESCRIBE HABITUALMENTE	COMO CORRESPONDE EN UN INFORME TÉCNICO
«Nosotros decidimos usar SQLite porque nos pareció más fácil»	«Se seleccionó SQLite porque no requiere servidor y el volumen previsto no supera los 5.000 registros»
«El sistema quedó bastante completo»	«Se implementaron los nueve requerimientos priorizados como <i>Must</i> ; los dos <i>Could</i> quedaron fuera de alcance»
«La IA nos ayudó hartó»	«Se validaron once fragmentos sugeridos por la herramienta: seis se adoptaron, tres se modificaron y dos se descartaron»

Tres reglas: **impersonal** —«se implementó», no «implementamos»—, **afirmaciones con dato** y **ningún adjetivo sin medida**. «Robusto» y «amigable» no dicen nada si no van seguidos de un número.

ANTES DE ENTREGAR

Lista de verificación del documento



Contenido

- Cada requerimiento aparece en la matriz de trazabilidad
- Cada diagrama tiene su lectura escrita al lado
- Cada decisión técnica declara la alternativa descartada
- La tabla de uso de IA tiene la columna de fundamento llena
- Lo que quedó fuera de alcance está declarado



Forma

- Figuras y tablas numeradas, rotuladas y citadas en el texto antes de aparecer
- Índice con páginas, y páginas numeradas
- Redacción impersonal, sin primera persona
- Sin adjetivos sin medida
- Solo se incluye el fragmento de código que ilustra una decisión
- El documento abre correctamente en PDF

La prueba final: entregue el informe a un compañero de otro grupo. Si puede decir qué hace el sistema y por qué está construido así, cumple su función.

BLOQUE

12

El ejercicio profesional: responder por la solución

El equipo no ejecuta una lista de tareas: asume la responsabilidad de que la solución resuelva el problema del cliente.

Hacer ingeniería es responder por lo que se construye: por qué se decidió así, qué riesgo se asumió y quién lo sostiene cuando falla.

Idea central del bloque

EL ENCARGO

Qué se espera de un consultor

☰ Un proveedor

Entrega lo que le pidieron, tal como se lo pidieron. Si lo pedido no resuelve el problema, no es asunto suyo.

«Usted me pidió un listado de destinos. Ahí está.»

🛡️ Un consultor

Entrega lo que **resuelve el problema**, y explica por qué esa es la forma correcta de resolverlo.

«El listado está, y además no permite borrar un destino que está dentro de un paquete vendido, porque eso dejaría reservas sin contenido.»

La diferencia no es el código: es que el segundo **puede sostener cada decisión**. Esa es exactamente la capacidad que se mide al final de esta unidad.

LEER AL CLIENTE

Lo que dice, lo que necesita y lo que no dice

LO QUE DIJO EL CLIENTE	LO QUE EN REALIDAD NECESITA	LO QUE NO DIJO, Y HAY QUE PREGUNTAR
«El cupo lo llevo en la cabeza y en el cuaderno»	Que el sistema controle el cupo, no que lo muestre	Qué hacer con las reservas que ya excedieron el cupo
«No borro los destinos porque están en paquetes vendidos»	Conservar la historia sin ensuciar el catálogo	Si un destino no disponible debe seguir visible para el cliente
«Me preocupa el RUT y el teléfono»	Que esos datos dejen de circular por correo	Quién debe poder verlos dentro del sistema
«Tengo que revisar el cuaderno hoja por hoja»	Consultar el historial en segundos	Cuánta historia hay que migrar del cuaderno

La tercera columna es la que distingue el trabajo profesional. Cuando no hay a quién preguntar —que es el caso aquí—, esa pregunta se convierte en un **supuesto declarado**.

CÓMO SE PRESENTA UN HALLAZGO

Cinco partes, siempre en el mismo orden

#	PARTE	QUÉ CONTIENE	EJEMPLO DEL CASO
1	El hallazgo	Qué se observó, en una frase y sin adjetivos	Seis reservas de la temporada superaron el cupo del paquete
2	La evidencia	De dónde sale el dato	La planilla de reservas y la entrevista del socio a cargo
3	El impacto	Qué cuesta que siga ocurriendo	Dos clientes quedaron fuera después de comprar su pasaje
4	La recomendación	Qué se propone, de forma concreta	Que el sistema calcule el cupo disponible y rechace la reserva que lo supere, informando el motivo
5	El esfuerzo	Cuánto cuesta y en qué iteración cabe	Dos horas, en la primera iteración

Una recomendación sin las partes 2, 3 y 5 no se puede decidir: el cliente no sabe si creerle, cuánto le importa ni cuánto le cuesta.

Un sistema no se entrega: **se explica y se sostiene**

Lo que distingue a un profesional no es la cantidad de código, sino que cada decisión tenga detrás a alguien capaz de fundamentarla.

TI3V21 • Programación Orientada a Objeto Seguro • INACAP Valparaíso